CSE480

Machine Vision Project

NAME	ID
ADHAM WALEED GAMAL	2100451
OMAR EMAD EL DIN HASSAN	2100580
OMAR KHALED AHMED	2100705
MARWAN MAHMOUD ALI	2100771
MOHAMED ALAA ABD EL KAREEM	2100392

Submitted to:

Prof. Hossam Hassan

Eng. Dina Zakaria

Table of Contents

1. Introduction & Problem Definition

This project centers on developing a robust **Action Recognition** system to automatically classify human activities within video sequences, a critical task in machine vision. Our solution employs a powerful hybrid **CNN-LSTM** deep learning architecture. This model first utilizes a pre-trained **MobileNetV2** (a Convolutional Neural Network) as a feature extractor, applied sequentially to each video frame using a **TimeDistributed** wrapper. This extracts spatial information (what is in the frame). The resulting sequence of features is then processed by two stacked **LSTM** (Long Short-Term Memory) layers, which are specialized recurrent networks designed to capture the temporal dependencies and context across the time axis (how the spatial features change over time).

The system is trained on extracted frames from video clips representing eight diverse actions (e.g., Diving, Basketball, Tai Chi). The data pipeline includes crucial preprocessing steps: videos are sampled into fixed-length frame sequences ($T=15$), resized to **224×224 pixels**, and enhanced using various **data augmentation** techniques (flipping, rotation, brightness) for model generalization. Finally, the frames are normalized using **MobileNetV2's specific preprocessing** before being fed to the model, which is optimized using Stochastic Gradient Descent (SGD) to classify the observed sequence into one of the designated action categories.

2. Data Preparation & Preprocessing

We implemented a systematic preprocessing pipeline to prepare video data for action recognition tasks. The goal was to standardize the videos, extract meaningful frames, and augment the data to improve model robustness. The preprocessing steps can be summarized as follows:

1. Frame Extraction

Videos were first processed to extract a fixed number of frames (e.g., 32 frames) per video. This ensures that all videos, regardless of their length or frame rate, are represented consistently in the dataset.

For instance, a 10-second video at 30 frames per second contains 300 frames, but we only extract 32 evenly spaced frames to reduce computational load while retaining temporal information. The frames were also resized to a standard resolution (224×224 pixels) to make them compatible with commonly used convolutional neural networks.

2. Data Augmentation

To improve the model's ability to generalize and handle variations in real-world videos, we applied a series of **data augmentations**. The augmentations were chosen

randomly for each video but applied consistently to all frames within that video. The augmentations included:

- **Horizontal Flip:** Randomly flips frames left-to-right to simulate mirrored actions.
  Example: If a basketball player is dribbling with the right hand, flipping produces an image of dribbling with the left hand.
- **Brightness Adjustment:** Slightly increases or decreases frame brightness to handle variations in lighting conditions.
  Example: A dimly lit classroom or bright outdoor environment is simulated.
- **Rotation:** Frames are rotated by a small angle (up to ±15°) to account for camera tilt or varying viewing angles.
- **Gaussian Blur:** Applies a mild blur to reduce sensitivity to small high-frequency details and simulate camera focus variations.
- **Gaussian Noise:** Adds random pixel noise to mimic sensor noise in real video captures.

By applying these augmentations consistently across all frames of a video, we ensure temporal consistency, which is critical for video-based action recognition.

3. Organized Storage

Each processed video was stored in a dedicated folder named after the original video. This hierarchical structure, grouped by action category (e.g., Basketball, Diving, Tai Chi), simplifies dataset management and facilitates downstream tasks such as batch loading during model training.

4. Benefits of This Preprocessing

The preprocessing pipeline ensures:

- **Consistency:** All videos have the same number of frames and resolution.
- **Robustness:** Augmentation introduces variations that improve model generalization.
- **Efficiency:** Extracting fewer, representative frames reduces computational requirements without losing important temporal information.

3. Methods and Algorithms

3.1. CNN-based Feature Extraction

To be able to recognize actions we need well defined features to be extracted from each frame. In order to achieve this, we utilized MobileNet, a pre-trained Convolutional Neural Network. By removing the output layer of the model, we can access the 1280x1 feature vector that it extracts from the image.

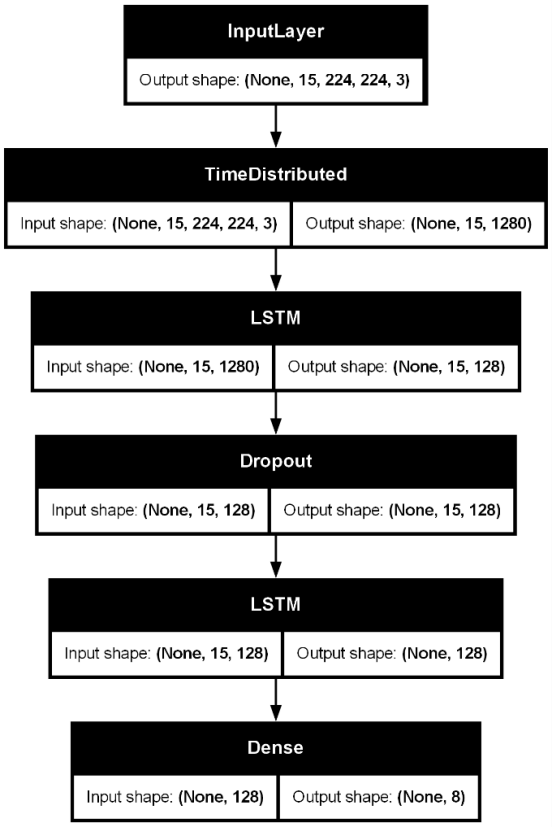
3.2. LSTM Network

Action Recognition is different from other basic Computer Vision tasks since single frames/images never contain sufficient information to recognize what action is being performed. Which is why it is necessary to utilize methods like LSTM networks that would maintain a memory of previously fed images which directly influence the

decision for the recognized action. This enables inference on sequences of frames and recognizing actions that take place over time.

3.3. Architecture Overview

Our architecture utilizes the pre-trained MobileNet model for feature extractions, we freeze its weights maintaining the pretrained values, which significantly decreases the need for data, computation, and training time.



We utilize TensorFlow's TimeDistributed Layer to extract features from all of the given frames simultaneously then the extracted 1280x1 feature vectors of each frame are collected in a Sequence and are then fed to 2 LSTM Layers each containing 128 units with a dropout rate of 0.5 between them to avoid overfitting and allow the model to detect more complex patterns in the data.

The output layer has a softmax activation function in order to output an estimate for the recognized action class

Layer (type)	Output Shape	Param #
video_input (InputLayer)	(None, 15, 224, 224, 3)	0
time_distributed_cnn (TimeDistributed)	(None, 15, 1280)	2,257,984
lstm_layer_1 (LSTM)	(None, 15, 128)	721,408
dropout (Dropout)	(None, 15, 128)	0
lstm_layer_2 (LSTM)	(None, 128)	131,584
classification_output (Dense)	(None, 8)	1,032

Total params: 3,112,008 (11.87 MB)
Trainable params: 854,024 (3.26 MB)
Non-trainable params: 2,257,984 (8.61 MB)

4. Experimental Results

4.1. Training analytics

4.1.1. Adam

4.1.1.1. Training Progress

Loss, Accuracy, Time, Validation Accuracy & Validation Loss

```
Epoch 1/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 252s 4s/step - accuracy: 0.5109 - loss: 1.7464 - val_accuracy: 0.8750 - val_loss: 1.2830
Epoch 2/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 71s 2s/step - accuracy: 0.9076 - loss: 0.9502 - val_accuracy: 0.9514 - val_loss: 0.5836
Epoch 3/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 71s 2s/step - accuracy: 0.9524 - loss: 0.4143 - val_accuracy: 0.9722 - val_loss: 0.2572
Epoch 4/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 71s 2s/step - accuracy: 0.9891 - loss: 0.1759 - val_accuracy: 0.9792 - val_loss: 0.1381
Epoch 5/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 71s 2s/step - accuracy: 0.9986 - loss: 0.0834 - val_accuracy: 0.9792 - val_loss: 0.1065
Epoch 6/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 0.9986 - loss: 0.0465 - val_accuracy: 0.9792 - val_loss: 0.0738
Epoch 7/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 0.9986 - loss: 0.0312 - val_accuracy: 0.9861 - val_loss: 0.0496
Epoch 8/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 1.0000 - loss: 0.0205 - val_accuracy: 0.9792 - val_loss: 0.0565
Epoch 9/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 1.0000 - loss: 0.0152 - val_accuracy: 0.9861 - val_loss: 0.0530
Epoch 10/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 1.0000 - loss: 0.0118 - val_accuracy: 0.9931 - val_loss: 0.0385
Epoch 11/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 1.0000 - loss: 0.0093 - val_accuracy: 0.9792 - val_loss: 0.0512
Epoch 12/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 1.0000 - loss: 0.0078 - val_accuracy: 0.9792 - val_loss: 0.0503
Epoch 13/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 1.0000 - loss: 0.0067 - val_accuracy: 0.9861 - val_loss: 0.0486
Epoch 14/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 73s 2s/step - accuracy: 1.0000 - loss: 0.0061 - val_accuracy: 0.9861 - val_loss: 0.0482
Epoch 15/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 78s 2s/step - accuracy: 1.0000 - loss: 0.0049 - val_accuracy: 0.9861 - val_loss: 0.0486
```

Testing results

```
--- Evaluating on Test Set ---
10/10 ━━━━━━━━━━━━━━━━━━━━ 30s 3s/step - accuracy: 0.9812 - loss: 0.0609
Final Test Loss: 0.0609
Final Test Accuracy: 98.12%
```

Performance per class:

```
--- Detailed Classification Report ---
10/10 ━━━━━━━━━━━━━━━━━━━━ 27s 1s/step

Classification Report (Per-Class Metrics):
                precision    recall  f1-score   support

    Basketball       1.00      1.00      1.00        20
   CliffDiving       0.91      1.00      0.95        21
       Diving        1.00      1.00      1.00        23
       Haircut       1.00      1.00      1.00        19
MilitaryParade       0.95      1.00      0.97        19
     SalsaSpin       1.00      0.90      0.95        20
        TaiChi       1.00      0.93      0.97        15
WritingOnBoard       1.00      1.00      1.00        23

      accuracy                           0.98       160
     macro avg       0.98      0.98      0.98       160
  weighted avg       0.98      0.98      0.98       160
```

Confusion Matrix:

```
Confusion Matrix:
[[20  0  0  0  0  0  0  0]
 [ 0 21  0  0  0  0  0  0]
 [ 0  0 23  0  0  0  0  0]
 [ 0  0  0 19  0  0  0  0]
 [ 0  0  0  0 19  0  0  0]
 [ 0  1  0  0  1 18  0  0]
 [ 0  1  0  0  0  0 14  0]
 [ 0  0  0  0  0  0  0 23]]
```

4.1.2. Adagrad

4.1.2.1. Training Progress

Loss, Accuracy, Time, Validation Accuracy & Validation Loss

```
Epoch 1/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 148s 2s/step - accuracy: 0.6345 - loss: 1.4809 - val_accuracy: 0.9028 - val_loss: 0.7881
Epoch 2/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 0.9416 - loss: 0.5436 - val_accuracy: 0.9792 - val_loss: 0.2887
Epoch 3/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 0.9769 - loss: 0.2240 - val_accuracy: 0.9444 - val_loss: 0.2222
Epoch 4/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 73s 2s/step - accuracy: 0.9932 - loss: 0.1153 - val_accuracy: 0.9861 - val_loss: 0.1004
Epoch 5/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 73s 2s/step - accuracy: 0.9986 - loss: 0.0598 - val_accuracy: 0.9931 - val_loss: 0.0720
Epoch 6/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 73s 2s/step - accuracy: 1.0000 - loss: 0.0374 - val_accuracy: 0.9931 - val_loss: 0.0660
Epoch 7/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 73s 2s/step - accuracy: 1.0000 - loss: 0.0282 - val_accuracy: 0.9931 - val_loss: 0.0560
Epoch 8/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 73s 2s/step - accuracy: 1.0000 - loss: 0.0225 - val_accuracy: 0.9931 - val_loss: 0.0553
Epoch 9/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 1.0000 - loss: 0.0182 - val_accuracy: 0.9931 - val_loss: 0.0533
Epoch 10/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 73s 2s/step - accuracy: 1.0000 - loss: 0.0149 - val_accuracy: 0.9931 - val_loss: 0.0493
Epoch 11/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 1.0000 - loss: 0.0133 - val_accuracy: 0.9931 - val_loss: 0.0494
Epoch 12/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 1.0000 - loss: 0.0117 - val_accuracy: 0.9861 - val_loss: 0.0500
Epoch 13/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 1.0000 - loss: 0.0104 - val_accuracy: 0.9931 - val_loss: 0.0497
Epoch 14/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 1.0000 - loss: 0.0090 - val_accuracy: 0.9931 - val_loss: 0.0472
Epoch 15/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 1.0000 - loss: 0.0080 - val_accuracy: 0.9931 - val_loss: 0.0475
```

Test:

```
--- Evaluating on Test Set ---
10/10 ━━━━━━━━━━━━━━━━━━━━ 13s 1s/step - accuracy: 0.9625 - loss: 0.1054
Final Test Loss: 0.1054
Final Test Accuracy: 96.25%
```

Per Class:

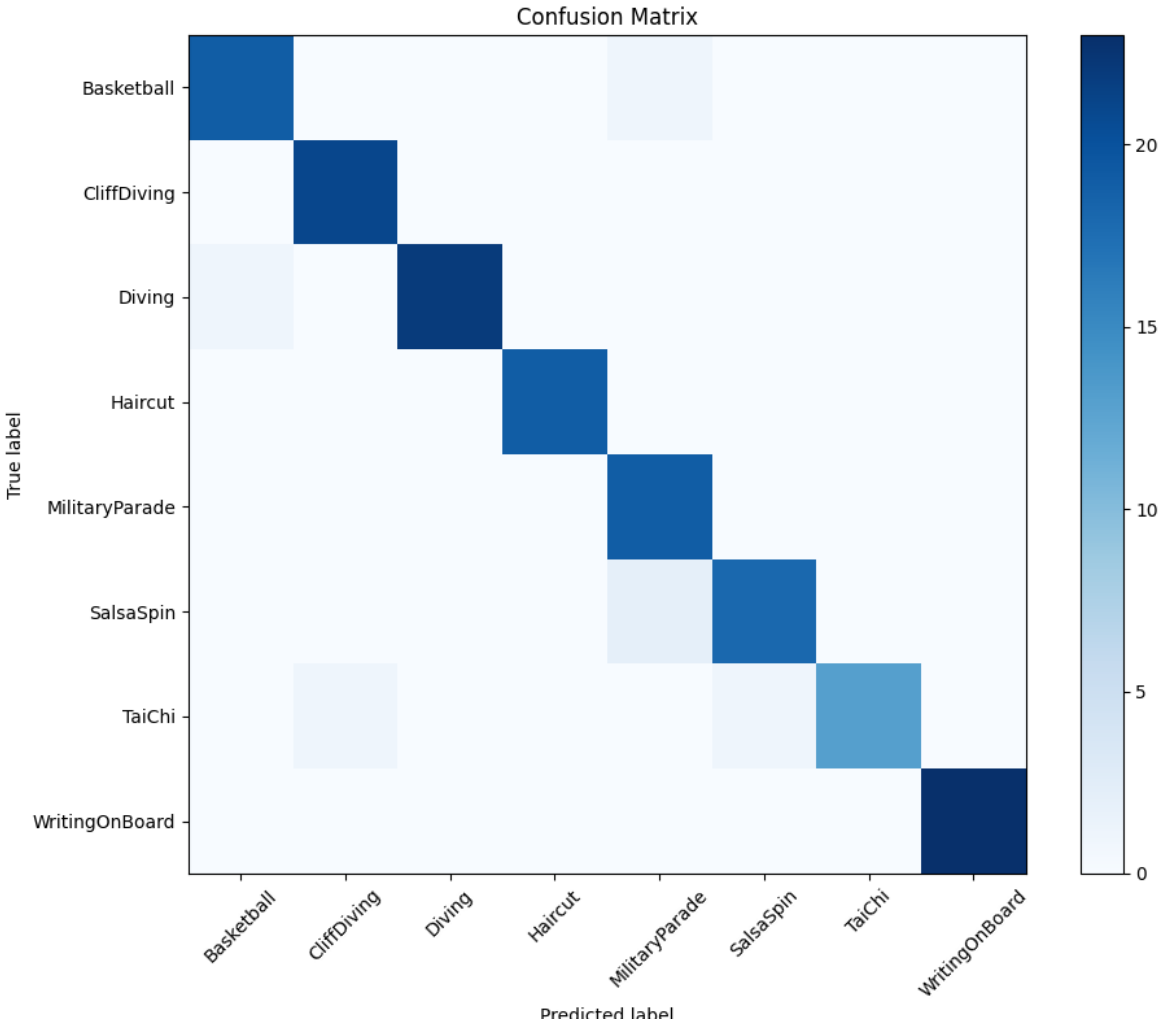
```
--- Detailed Classification Report ---
10/10 ━━━━━━━━━━━━━━━━━━━━ 26s 1s/step

Classification Report (Per-Class Metrics):
                precision    recall  f1-score   support

    Basketball       0.95      0.95      0.95        20
   CliffDiving       0.95      1.00      0.98        21
       Diving        1.00      0.96      0.98        23
       Haircut       1.00      1.00      1.00        19
MilitaryParade       0.86      1.00      0.93        19
     SalsaSpin       0.95      0.90      0.92        20
        TaiChi       1.00      0.87      0.93        15
WritingOnBoard       1.00      1.00      1.00        23

      accuracy                           0.96       160
     macro avg       0.96      0.96      0.96       160
  weighted avg       0.97      0.96      0.96       160
```

Confusion Matrix:



4.1.3. SGD

4.1.3.1. Training Progress

Loss, Accuracy, Time, Validation Accuracy & Validation Loss

```
Epoch 1/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 147s 2s/step - accuracy: 0.4198 - loss: 1.8491 - val_accuracy: 0.6875 - val_loss: 1.5570
Epoch 2/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 73s 2s/step - accuracy: 0.7894 - loss: 1.3317 - val_accuracy: 0.8958 - val_loss: 1.0649
Epoch 3/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 73s 2s/step - accuracy: 0.8845 - loss: 0.8981 - val_accuracy: 0.9306 - val_loss: 0.7154
Epoch 4/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 240s 5s/step - accuracy: 0.9321 - loss: 0.5990 - val_accuracy: 0.9514 - val_loss: 0.4699
Epoch 5/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 345s 8s/step - accuracy: 0.9660 - loss: 0.4068 - val_accuracy: 0.9444 - val_loss: 0.3576
Epoch 6/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 342s 7s/step - accuracy: 0.9851 - loss: 0.2819 - val_accuracy: 0.9653 - val_loss: 0.2525
Epoch 7/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 340s 7s/step - accuracy: 0.9905 - loss: 0.2075 - val_accuracy: 0.9722 - val_loss: 0.1978
Epoch 8/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 317s 7s/step - accuracy: 0.9959 - loss: 0.1531 - val_accuracy: 0.9722 - val_loss: 0.1562
Epoch 9/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 70s 2s/step - accuracy: 0.9959 - loss: 0.1202 - val_accuracy: 0.9792 - val_loss: 0.1294
Epoch 10/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 70s 2s/step - accuracy: 0.9986 - loss: 0.0974 - val_accuracy: 0.9722 - val_loss: 0.1260
Epoch 11/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 70s 2s/step - accuracy: 0.9986 - loss: 0.0792 - val_accuracy: 0.9792 - val_loss: 0.1055
Epoch 12/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 71s 2s/step - accuracy: 0.9986 - loss: 0.0677 - val_accuracy: 0.9722 - val_loss: 0.1023
Epoch 13/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 72s 2s/step - accuracy: 1.0000 - loss: 0.0567 - val_accuracy: 0.9722 - val_loss: 0.0919
Epoch 14/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 71s 2s/step - accuracy: 0.9986 - loss: 0.0488 - val_accuracy: 0.9792 - val_loss: 0.0899
Epoch 15/15
46/46 ━━━━━━━━━━━━━━━━━━━━ 71s 2s/step - accuracy: 1.0000 - loss: 0.0439 - val_accuracy: 0.9722 - val_loss: 0.0878
```

Test:

```
--- Evaluating on Test Set ---
10/10 ━━━━━━━━━━━━━━━━━━━━ 13s 1s/step - accuracy: 0.9750 - loss: 0.0872
Final Test Loss: 0.0872
Final Test Accuracy: 97.50%
```

Per Class:

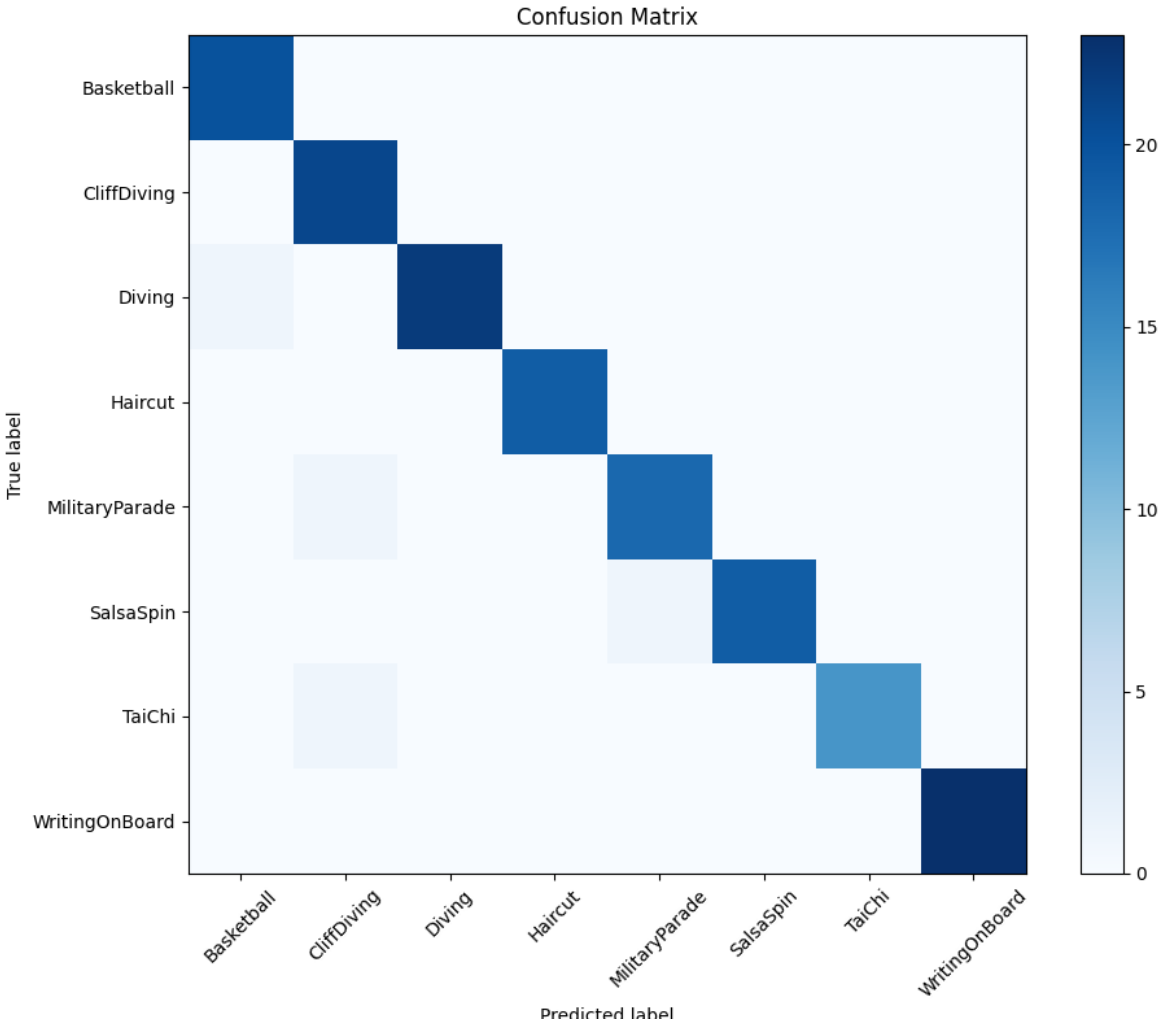
```
--- Detailed Classification Report ---
10/10 ━━━━━━━━━━━━━━━━━━━━ 26s 1s/step

Classification Report (Per-Class Metrics):
                precision    recall  f1-score   support

    Basketball       0.95      1.00      0.98        20
   CliffDiving       0.91      1.00      0.95        21
       Diving        1.00      0.96      0.98        23
      Haircut        1.00      1.00      1.00        19
MilitaryParade       0.95      0.95      0.95        19
     SalsaSpin       1.00      0.95      0.97        20
       TaiChi        1.00      0.93      0.97        15
WritingOnBoard       1.00      1.00      1.00        23

     accuracy                           0.97       160
    macro avg       0.98      0.97      0.97       160
 weighted avg       0.98      0.97      0.98       160
```

Confusion Matrix:



Comments

We were able to achieve great results using our architecture with great performance across all classes. As for optimizers Adam and Adagrad performed great with training time being much faster that standard SGD.